

Esercizio di programmazione concorrente

Si consideri un sistema produttore–consumatore, nel quale si producono e consumano oggetti di classe Roba, contenenti un attributo qualità. Produttori e consumatori usano un deposito in cui vengono memorizzati gli oggetti istanze di Roba. Il deposito è un buffer di capienza limitata. I produttori producono oggetti di qualità variabile e li inseriscono nel deposito. I consumatori cercano di consumare oggetti aventi qualità non inferiore a un certo valore.

Nel codice dato le operazioni di inserimento ed estrazione di oggetti dal deposito hanno successo o falliscono immediatamente, e i produttori e consumatori si comportano di conseguenza. In particolare, se un consumatore non trova un oggetto con la qualità desiderata, prova a estrarne uno di qualità un po' inferiore, iterativamente.

Si chiede di modificare il codice dato in modo che

- Le operazioni di inserimento risultino bloccanti (il thread viene messo in attesa) se il deposito è pieno.
- Le operazioni di estrazione risultino bloccanti se il deposito non contiene un oggetto avente la qualità desiderata, per un tempo massimo T. L'estrazione si può concludere con successo (viene restituito l'oggetto di qualità desiderata) o può fallire (il tempo T trascorre interamente senza che un oggetto di qualità adeguata sia disponibile). In caso di fallimento, il consumatore si comporta come nel codice dato, provando ad estrarre un oggetto di qualità inferiore.

Il codice prodotto deve evitare corse critiche, deadlock, starvation, e altri problemi tipici della programmazione concorrente.

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio di programmazione distribuita (con uso di Socket)

Si desidera un sistema distribuito che funziona come il sistema descritto nell'esercizio di programmazione concorrente, ma in cui il deposito, i produttori e i consumatori possono risiedere su macchine diverse. Il sistema deve essere realizzato usando socket per la comunicazione. Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.

Esercizio di programmazione distribuita (con uso di RMI)

Si desidera un sistema distribuito che funziona come il sistema descritto nell'esercizio di programmazione concorrente, ma in cui il deposito, i produttori e i consumatori possono risiedere su macchine diverse. Il sistema deve essere realizzato usando RMI. Come sempre, occorre evitare i problemi tipici della programmazione concorrente (corse critiche, deadlock, starvation, ecc.).

Si ricorda che bisogna consegnare i file .java (non i .class!), raggruppati in un unico file zip.